SPÉCIFICATION TECHNIQUE & ARCHITECTURE ALGORITHMIQUE

SIMULATION ET ASSERVISSEMENT D'UN PENDULE INVERSÉ

Cahier des Charges, Modélisation & Préconception (TAD / Signatures) — Cible : Langage C (ISO C99)

01 Cahier des Charges & Périmètre Fonctionnel

Le projet vise à concevoir un moteur de simulation physique déterministe reproduisant la dynamique exacte d'un **pendule articulé sur un chariot motorisé** (système *Cart-Pole*), suivi d'un module d'automatique capable d'amener et de stabiliser le pendule à son équilibre vertical haut (θ = 0 rad) par le seul déplacement horizontal du chariot. L'architecture sépare strictement la logique mathématique et décisionnelle de la couche d'interface graphique.

Jalon / Module	Inclus dans le Périmètre (In-Scope)	Exclus du Périmètre (Out-of-Scope)
1. Moteur Physique	Équations d'Euler-Lagrange non-linéaires couplées (grands angles), gravité g, inerties couplées chariot/pendule.	Déformation élastique de la tige, traînée aérodynamique quadratique turbulente.
2. Liaisons & Pertes	Liaison glissière (rail 1D, frottement visqueux b) et liaison pivot sur chariot (amortissement fluide c).	Frottement sec de Coulomb discontinu (stick-slip), jeu mécanique d'engrenage.
3. Solveur Numérique	Intégration Runge-Kutta d'ordre 4 (RK4) à pas fixe (Δt = 1 ms), normalisation angulaire dans]−π, π].	Solveurs à pas adaptatif (RK45), intégration d'Euler explicite (instable).
4. Asservissement	Pompage énergétique (*Swing-up* depuis θ = π), stabilisation linéaire (retour d'état LQR/PID) et saturation F_{max}.	Modélisation électrique interne du moteur CC (inductance, hacheur PWM).
5. Entrées / Sorties	Contrat d'interface abstrait pour le rendu 2D temps-réel (60 Hz) et l'export télémétrique CSV.	Moteur 3D complexe, interface graphique lourde couplée au solveur.

02 Modélisation Physique et Justification des Liaisons

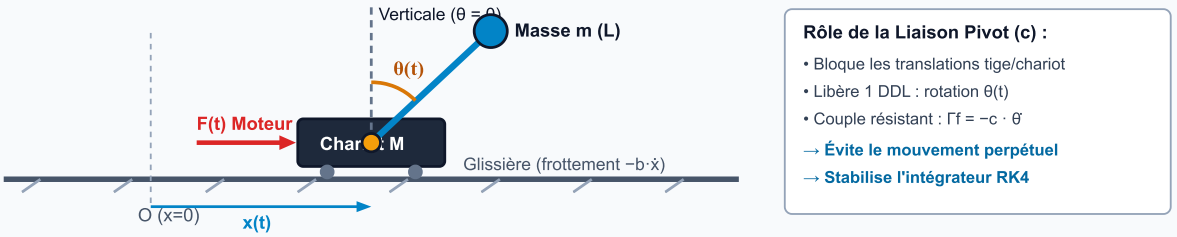


Figure 1 — Paramétrage mécanique du pendule sur chariot et modélisation de l'articulation pivot.

Pourquoi modéliser une liaison pivot avec amortissement fluide (c) ?

En mécanique analytique, la **liaison pivot** matérialise l'axe de rotation (roulement à billes) reliant la base du pendule au chariot : elle impose à la base de la tige de subir exactement l'accélération horizontale $\ddot{x}$ du chariot tout en autorisant la libre rotation d'angle θ. L'ajout d'un **coefficient d'amortissement visqueux c** (générant un couple opposé à la vitesse angulaire $\Gamma_f = -c \cdot \dot{\theta}$) modélise le frottement du roulement et la résistance de l'air. Sans ce terme (c = 0), le pendule ne s'arrêterait jamais d'osciller en boucle ouverte et les micro-erreurs numériques d'arrondi ne seraient jamais dissipées.

03 Formulation Mathématique pour l'Intégration Numérique

L'application des équations d'Euler-Lagrange au système couplé fournit deux équations différentielles du second ordre. Pour les résoudre numériquement via l'algorithme de **Runge-Kutta d'ordre 4 (RK4)**, on isole explicitement les accélérations $\ddot{x}$ et $\ddot{\theta}$ en fonction du vecteur d'état courant $Y = (x, \dot{x}, \theta, \dot{\theta})^T$ et de la force moteur F :

$$D(\theta) = M + m \cdot \sin^2(\theta) \quad | \quad \ddot{x} = [\, F - b\cdot\dot{x} + m\cdot L\cdot\dot{\theta}^2\cdot\sin(\theta) - m\cdot g\cdot\sin(\theta)\cos(\theta) + (c\cdot\dot{\theta}\cdot\cos(\theta)) / L \,] / D(\theta)$$

$$\ddot{\theta} = [\, g\cdot\sin(\theta) - \ddot{x}\cdot\cos(\theta) - c\cdot\dot{\theta} / (m\cdot L) \,] / L$$

Loi de Commande Hybride (Passage de θ = π à θ = 0)

Le système étant fortement non-linéaire ($\cos(\theta)$ change de signe entre le bas et le haut), l'asservissement opère selon une machine à états : **(1) Mode Swing-Up** lorsque $|\theta| > \theta_{seuil}$ (env. 0,35 rad), où le chariot pompe de l'énergie mécanique selon le signe de $\dot{\theta}\cdot\cos(\theta)$; **(2) Mode Stabilisation** lorsque $|\theta| \leq \theta_{seuil}$, où un retour d'état linéaire $F = K_x(x - x_{ref}) + K_v\cdot\dot{x} + K_\theta\cdot\theta + K_\omega\cdot\dot{\theta}$ verrouille le pendule à la verticale tout en ramenant le chariot au centre.

04 Analyse Descendante Complète (Top-Down Design)

L'analyse descendante décompose le problème global (*Simuler et Asservir le Pendule*) en sous-problèmes hiérarchiques indépendants jusqu'à obtenir des briques élémentaires directement traduisibles en procédures et fonctions.

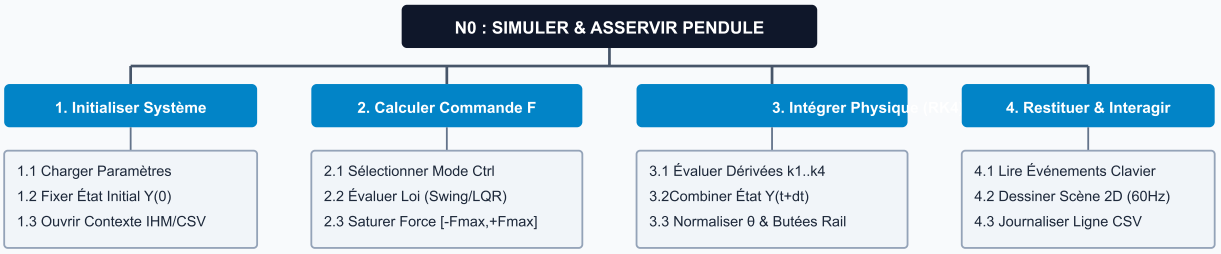


Figure 2 — Décomposition descendante modulaire du programme de simulation.

Branche	Responsabilité Logique	Données Entrantes (In)	Données Sortantes (Out)
1. Initialisation	Instancier les constantes physiques, l'état t=o, les gains et l'affichage.	Constantes de configuration, dimensions fenêtre, chemin CSV.	Structures `Parametres`, `Etat`, `Controleur`, `Affichage`.
2. Commande	Décider du mode d'action et calculer la poussée horizontale optimale.	`EtatSysteme` courant, `ParametresPhysiques`, consigne x_{ref}.	Effort moteur borné $F \in [-F_{max}, +F_{max}]$, mode actif.
3. Physique & RK4	Faire progresser le vecteur d'état de t à $t + \Delta t$ selon la mécanique.	`EtatSysteme` à t, effort appliqué F, pas Δt.	Nouvel `EtatSysteme` à $t + \Delta t$ avec angle normalisé.
4. Restitution (I/O)	Capter les perturbations utilisateur, afficher le système et archiver.	`EtatSysteme`, force F, énergies (E_c, E_p), horloge t.	Trame graphique 2D, ligne CSV, commande d'arrêt ou perturbation.

05 Dossier de Préconception : Types Abstraits de Données (TAD)

Avant toute implémentation en C, les structures de données sont formalisées sous forme de **Types Abstraits de Données (TAD)** en pseudocode algorithmique. Aucune variable globale n'est tolérée : l'intégralité du contexte est encapsulée dans ces enregistrements transmis par référence aux sous-programmes.

5.1 TAD du Noyau Physique et Cinématique

```
// Représente le vecteur d'état continu Y(t) du système dynamique à un instant donné
Type EtatSysteme = Enregistrement
    x : Réel // Position horizontale du chariot sur le rail (en m)
    v_x : Réel // Vitesse linéaire horizontale du chariot x_point (en m/s)
    theta : Réel // Angle du pendule / verticale haute (en rad, dans ]-π, π])
    v_theta : Réel // Vitesse angulaire du pendule theta_point (en rad/s)
FinEnregistrement
// Représente le vecteur dérivé dY/dt = (v_x, a_x, v_theta, a_theta) utilisé par RK4
Type DeriveeEtat = Enregistrement
    dx : Réel // Dérivée de la position (= v_x)
    dv_x : Réel // Accélération linéaire du chariot x_point_point (en m/s²)
    dtheta : Réel // Dérivée de l'angle (= v_theta)
    dv_theta : Réel // Accélération angulaire du pendule theta_point_point (en rad/s²)
FinEnregistrement
// Regroupe les invariants physiques du banc mécanique
Type ParametresPhysiques = Enregistrement
    masse_chariot : Réel // Masse M du chariot motorisé (en kg, > 0)
    masse_pendule : Réel // Masse m concentrée à l'extrémité de la tige (en kg, > 0)
    longueur_tige : Réel // Longueur L de la tige rigide (en m, > 0)
    gravite : Réel // Accélération de la pesanteur g (9.81 m/s²)
    frott_chariot : Réel // Coefficient visqueux b de la liaison glissière (en N.s/m, >= 0)
    frott_pivot : Réel // Coefficient d'amortissement fluide c du pivot (en N.m.s/rad, >= 0)
    force_max : Réel // Saturation de force F_max du moteur (en N, > 0)
    limite_rail : Réel // Demi-course utile maximale X_max sur le rail (en m, > 0)
FinEnregistrement
```

5.2 TAD du Module d'Automatique et d'Asservissement

```
// Machine à états finis régissant la stratégie de contrôle
Type ModeControle = Énumération
    MODE_BOUCLE_OUVERTE, // Moteur débrayé (F = 0), évolution libre sous gravité
    MODE_SWING_UP, // Pompage non-linéaire d'énergie depuis le bas (|theta| > seuil)
    MODE_STABILISATION // Régulation linéaire autour de la verticale haute (|theta| <= seuil)
FinÉnumération
Type Regulateur = Enregistrement
    mode_actif : ModeControle // État courant de la machine à états
    gains_etat : Tableau[1..4] de Réel // Vecteur de gains [K_x, K_vx, K_theta, K_vtheta]
    gain_swing : Réel // Gain énergétique k_E du mode Swing-Up
    seuil_bascule : Réel // Angle critique d'accrochage linéaire (ex: 0.35 rad)
    consigne_x : Réel // Position cible x_ref du chariot sur le rail (m)
    integrale_err : Réel // Accumulateur intégral pour éliminer l'erreur statique
FinEnregistrement
```

5.3 TAD de l'Affichage, des Entrées et de la Simulation

```
Type ActionUtilisateur = Énumération
    ACTION_AUCUNE, ACTION_QUITTER, ACTION_REINITIALISER, ACTION_PERTURBER_GAUCHE,
    ACTION_PERTURBER_DROITE, ACTION_BASCULER_CONTROLE
FinÉnumération
Type ContexteExecution = Enregistrement
    temps_courant : Réel // Horloge physique simulée t (en s)
    dt_physique : Réel // Pas d'intégration RK4 (ex: 0.001 s = 1000 Hz)
    dt_affichage : Réel // Période de rafraîchissement écran (ex: 0.0166 s = 60 Hz)
    en_cours : Booléen // Drapeau de maintien de la boucle principale
FinEnregistrement
```

06 Préconception : Signatures des Modules Logiques

Les contrats des sous-programmes précisent explicitement le mode de passage de chaque paramètre : `[Entrée]` (lecture seule, équivalent à une valeur ou un pointeur `const` en C), `[Sortie]` (résultat produit par la procédure), ou `[Entrée/Sortie]` (état modifié sur place).

6.1 Module Physique (`Module_Physique`)

```
// Initialise les paramètres physiques avec des valeurs réalistes et vérifie leur cohérence Procédure InitialiserParametres( Sortie params : ParametresPhysiques ) // Pré-condition : Aucune // Post-condition : M > 0, m > 0, L > 0, g = 9.81, b >= 0, c >= 0 // Ramène n'importe quel angle réel dans l'intervalle principal ]-π, +π] Fonction NormaliserAngle( Entrée angle_brut : Réel ) : Réel // Évalue les équations d'Euler-Lagrange pour remplir le vecteur dérivé dY/dt Procédure CalculerDerivees( Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques, Entrée force : Réel, Sortie deriv : DeriveeEtat ) // Rôle : Calcule le dénominateur D = M + m*sin²(theta), résout a_x puis injecte a_x dans a_theta // Calcule l'énergie mécanique totale du système complet (Validation & Diagnostic) Fonction CalculerEnergieTotale( Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques ) : Réel // Calcule l'énergie mécanique du pendule seul par rapport à l'équilibre haut (E = 0 en θ = 0) Fonction CalculerEnergiePendule( Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques ) : Réel
```

6.2 Module Intégrateur Numérique (`Module_Integrateur`)

```
// Effectue une combinaison linéaire : etat_res = etat_base + facteur * deriv Procédure CombinerEtatIntermediaire( Entrée etat_base : EtatSysteme, Entrée deriv : DeriveeEtat, Entrée facteur : Réel, Sortie etat_res : EtatSysteme ) // Fait avancer l'état du système de t à t + dt via la méthode de Runge-Kutta d'ordre 4 Procédure IntegrerPasRK4( Entrée/Sortie etat : EtatSysteme, Entrée params : ParametresPhysiques, Entrée force : Réel, Entrée dt : Réel ) // Post-condition : etat est mis à jour à t+dt, etat.theta est normalisé dans ]-π, +π], // et les butées mécaniques du rail [-limite_rail, +limite_rail] sont appliquées.
```

6.3 Module Contrôleur & Asservissement (`Module_Controleur`)

```
// Borne une valeur réelle dans l'intervalle [val_min, val_max] Fonction SaturerValeur(Entrée val : Réel, Entrée val_min : Réel, Entrée val_max : Réel) : Réel // Met à jour la machine à états (Swing-Up vs Stabilisation) selon le cône d'angle courant Procédure ActualiserModeControle( Entrée/Sortie reg : Regulateur, Entrée etat : EtatSysteme ) // Loi de pompage énergétique (Swing-Up) lorsque le pendule est loin de la verticale haute Fonction CalculerForceSwingUp( Entrée reg : Regulateur, Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques ) : Réel // Loi de retour d'état linéaire (LQR / PID) au voisinage de theta = 0 Fonction CalculerForceStabilisation( Entrée/Sortie reg : Regulateur, Entrée etat : EtatSysteme, Entrée dt : Réel ) : Réel // Fonction maîtresse du contrôleur : sélectionne le mode, calcule la force et la sature Fonction CalculerCommandeMoteur( Entrée/Sortie reg : Regulateur, Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques, Entrée dt : Réel ) : Réel
```

07 Préconception : Signatures d'Affichage et Télémétrie

Bien que le code graphique puisse être généré séparément, son contrat d'interface est figé dès la préconception pour que la boucle logique puisse l'appeler sans connaître la bibliothèque sous-jacente (SDL2, Raylib ou export CSV).

```
// Initialise la fenêtre graphique et ouvre le fichier de journalisation CSV Procédure InitialiserInterface( Entrée largeur_px : Entier, Entrée hauteur_px : Entier, Entrée chemin_csv : Chaîne, Sortie succes : Booléen ) // Capture les entrées clavier/souris sans bloquer l'exécution de la boucle physique Fonction LireActionUtilisateur() : ActionUtilisateur // Applique une impulsion extérieure sur la vitesse angulaire ou réinitialise l'état Procédure TraiterInteraction( Entrée action : ActionUtilisateur, Entrée/Sortie etat : EtatSysteme, Entrée/Sortie reg : Regulateur, Entrée/Sortie exec : ContexteExecution ) // Rendu visuel d'une image : rail, chariot, tige, vecteur force et bandeau télémétrique Procédure DessinerEtatCourant( Entrée etat : EtatSysteme, Entrée params : ParametresPhysiques, Entrée reg : Regulateur, Entrée force : Réel, Entrée temps : Réel ) // Écrit une ligne horodatée (t, x, v_x, theta, v_theta, force, energie) dans le flux CSV Procédure EnregistrerMesureCSV( Entrée temps : Réel, Entrée etat : EtatSysteme, Entrée force : Réel, Entrée energie : Réel ) // Ferme proprement les descripteurs de fichiers et libère la mémoire graphique Procédure FermerInterface()
```

08 Orchestration Temporelle et Flux de Données

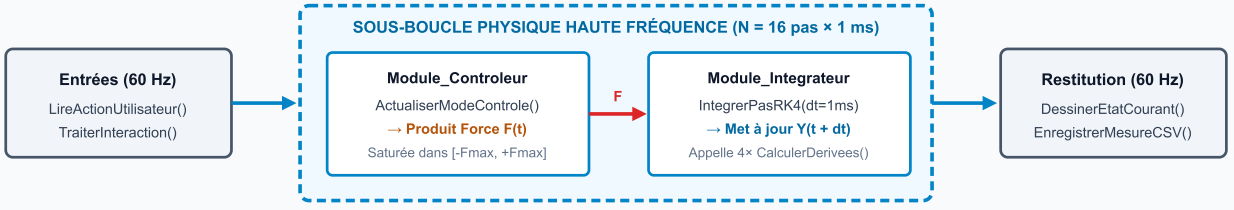


Figure 3 — Architecture de la boucle maîtresse à sous-échantillonnage physique (Sub-stepping).

8.1 Squelette Algorithmique du Programme Principal (Orchestrateur)

```
// Algorithme de niveau 0 orchestrant les appels sans exposer l'implémentation interne Algorithme SimulateurPenduleInverse Variables params : ParametresPhysiques ; etat : EtatSysteme ; reg : Regulateur exec : ContexteExecution ; act : ActionUtilisateur ; force_cmd, acum_temps : Réel ; ok : Booléen Début InitialiserParametres(params) InitialiserEtatSysteme(etat, 0.0, 3.14159) // Départ pendule en bas (theta = π) InitialiserRegulateur(reg, MODE_SWING_UP) InitialiserInterface(1280, 720, "telemetrie.csv", ok) acum_temps ← 0.0 TantQue (exec.en_cours = VRAI) Faire act ← LireActionUtilisateur() TraiterInteraction(act, etat, reg, exec) // Sous-boucle physique à pas fixe (dt_physique = 1 ms) jusqu'au prochain rafraîchissement 60 Hz TantQue (acum_temps < exec.dt_affichage) Faire force_cmd ← CalculerCommandeMoteur(reg, etat, params, exec.dt_physique) IntegrerPasRK4(etat, params, force_cmd, exec.dt_physique) exec.temps_courant ← exec.temps_courant + exec.dt_physique acum_temps ← acum_temps + exec.dt_physique FinTantQue acum_temps ← acum_temps - exec.dt_affichage DessinerEtatCourant(etat, params, reg, force_cmd, exec.temps_courant) EnregistrerMesureCSV(exec.temps_courant, etat, force_cmd, CalculerEnergieTotale(etat, params)) FinTantQue FermerInterface() Fin
```